



HAUTE ÉCOLE
D'INGÉNIERIE ET DE GESTION
DU CANTON DE VAUD
www.heig-vd.ch



Laboratoire 5 : Description d'éléments mémoire en VHDL

Département : Technologies de l'Information et de la Communication (TIC)
Unité d'enseignement : Conception de systèmes numériques (CSN)

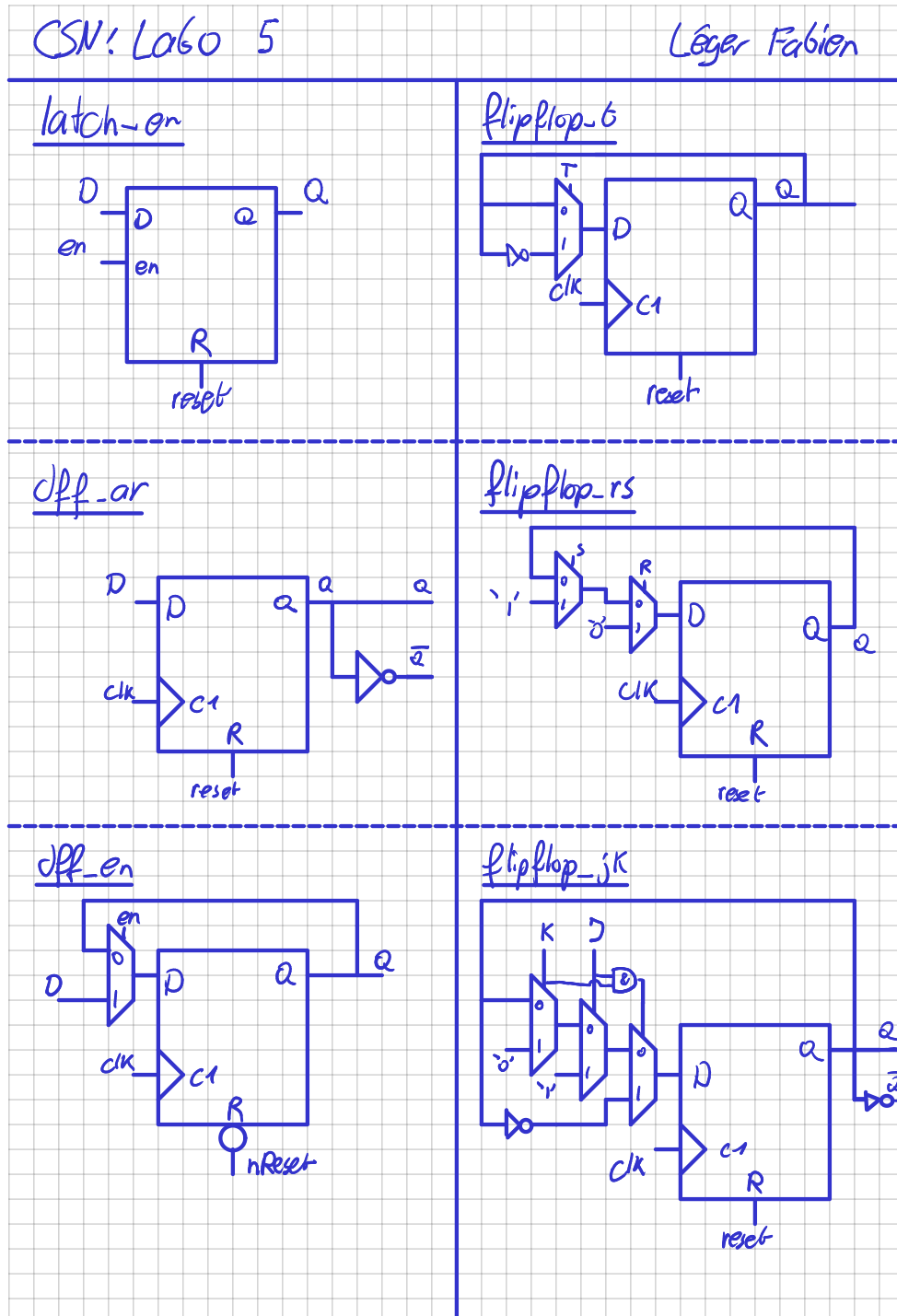
Auteurs : Léger Fabien
Professeur : E. Messerli
Assistant : B. Sutterlet
Classe : CSN-A
Date : 22 avril 2026

1 Introduction

Ce laboratoire a pour objectif de nous apprendre comment implémenter des éléments mémoires en VHDL grâce à l'instruction `process` permettant de faire du séquentiel. Celle-ci permet de réagir à des changements d'état de certaines entrées afin d'y réagir.

On pourra ainsi avoir des réactions sur des `clock` ou des `reset`. Il est également possible d'omettre le `else` dans un bloc d'instruction `if-else` ce qui dit au compilateur de mémoriser les sorties et garder les mêmes.

2 Analyse et conception



3 Réalisation et implémentation

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity dff_ar is
5     port(
6         clk_i    : in    std_logic;
7         reset_i   : in    std_logic;
8         D_i       : in    std_logic;
9         Q_o       : out   std_logic;
10        nQ_o      : out   std_logic
11    );
12 end dff_ar ;
13
14 architecture comport of dff_ar is
15
16     signal Q_s: std_logic;
17
18     begin
19
20         process(reset_i, clk_i)
21         begin
22             if reset_i = '1' then
23                 Q_s <= '0';
24             elsif rising_edge(clk_i) then
25                 Q_s <= D_i;
26             end if;
27         end process;
28
29         Q_o <= Q_s;
30         nQ_o <= not Q_s;
31
32     end comport;

```

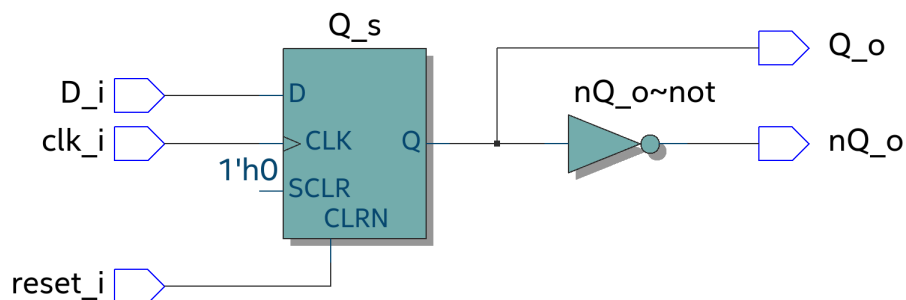


FIGURE 1 – Quartus Prime vue RTL dff_ar

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  entity dff_en is
5      port(
6          clk_i      : in      std_logic;
7          nReset_i   : in      std_logic;
8          D_i        : in      std_logic;
9          en_i       : in      std_logic;
10         Q_o        : out     std_logic
11     );
12 end dff_en ;
13
14
15 architecture comport of dff_en is
16
17 begin
18
19     process(nReset_i, clk_i)
20     begin
21         if nReset_i = '0' then
22             Q_o <= '0';
23         elsif rising_edge(clk_i) then
24             if en_i = '1' then
25                 Q_o <= D_i;
26             end if;
27         end if;
28     end process;
29
30 end comport;

```

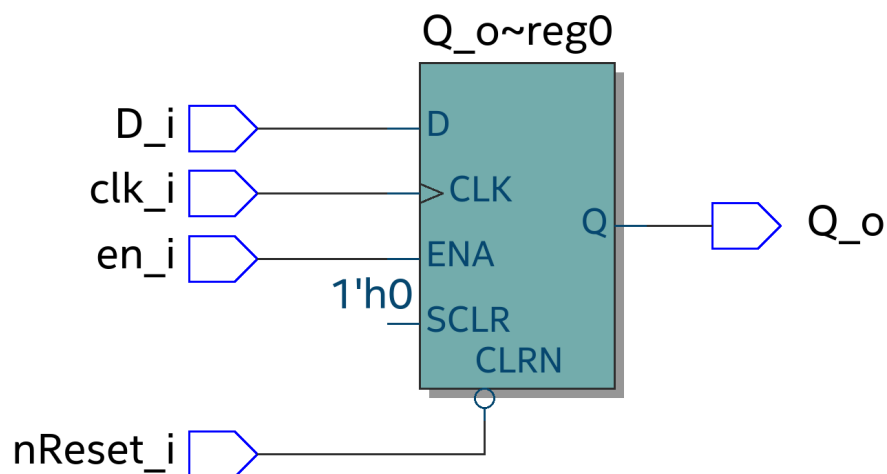


FIGURE 2 – Quartus Prime vue RTL dff_en

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity flipflop_t is
5     port(clk_i      : in      std_logic;
6          reset_i    : in      std_logic;
7          T_i        : in      std_logic;
8          Q_o        : out     std_logic
9     );
10 end flipflop_t ;
11
12
13 architecture comport of flipflop_t is
14     signal reset_s : std_logic;
15     signal q_s      : std_logic;
16
17 begin
18     -- Adaptation polarite
19     reset_s <= reset_i;
20     -- q_s = '-' otherwise ? => no need to affect q_s
21
22     process(reset_s, clk_i)
23     begin
24         if reset_s = '1' then
25             q_s <= '0';
26         elsif rising_edge(clk_i) then
27             if T_i = '1' then
28                 q_s <= not q_s;
29             end if;
30         end if;
31     end process;
32
33     Q_o <= q_s;
34
35 end comport;

```

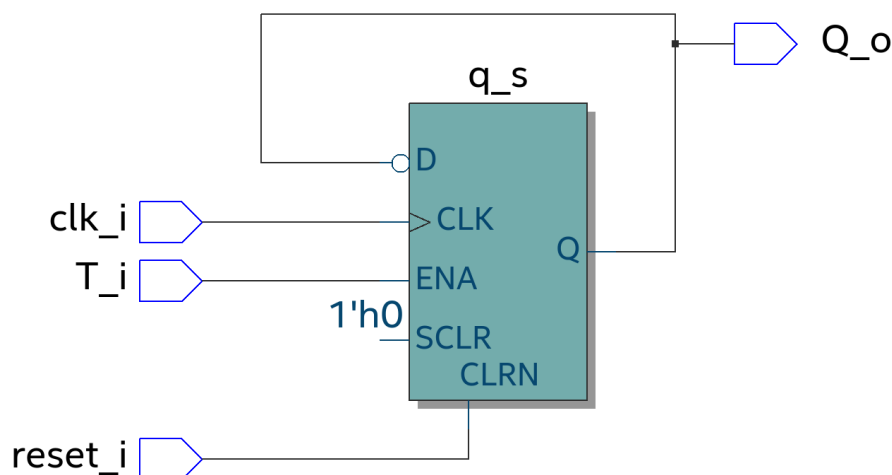


FIGURE 3 – Quartus Prime vue RTL flipflop_t

```

1 library ieee;
2   use ieee.std_logic_1164.all;
3
4 entity flipflop_rs is
5     port(clk_i      : in      std_logic;
6          reset_i    : in      std_logic;  --asynchrone
7          R_i        : in      std_logic;  --synchrone
8          S_i        : in      std_logic;  --synchrone
9          Q_o        : out     std_logic
10    );
11 end flipflop_rs ;
12
13 architecture comport of flipflop_rs is
14
15     signal q_s: std_logic;
16
17 begin
18     --Adaptation polarite
19
20     process(reset_i, clk_i)
21     begin
22         if reset_i = '1' then
23             q_s <= '0';
24         elsif rising_edge(clk_i) then
25             if R_i = '1' then
26                 q_s <= '0';
27             elsif S_i = '1' then
28                 q_s <= '1';
29             end if;
30         end if;
31     end process;
32
33     Q_o <= q_s;
34
35 end comport;

```

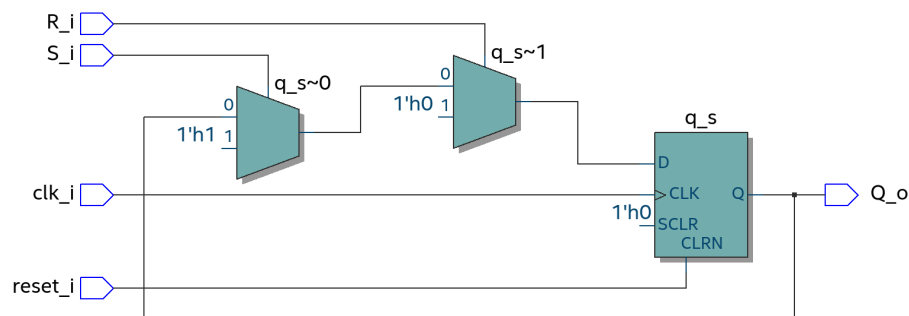


FIGURE 4 – Quartus Prime vue RTL flipflop_rs

```

1 library ieee;
2   use ieee.std_logic_1164.all;
3
4 entity flipflop_jk is
5     port(clk_i      : in      std_logic;
6          reset_i    : in      std_logic;
7          J_i        : in      std_logic;
8          K_i        : in      std_logic;
9          Q_o        : out     std_logic;
10         nQ_o       : out     std_logic
11     );
12 end flipflop_jk ;
13
14 architecture comport of flipflop_jk is
15
16     signal q_s: std_logic;
17
18 begin
19     --Adaptation polarite
20
21     process(reset_i, clk_i)
22     begin
23         if reset_i = '1' then
24             q_s <= '0';
25         elsif rising_edge(clk_i) then
26             if J_i = '1' and K_i = '1' then
27                 q_s <= not q_s;
28             elsif J_i = '1' then
29                 q_s <= '1';
30             elsif K_i = '1' then
31                 q_s <= '0';
32             end if;
33         end if;
34     end process;
35
36     Q_o <= q_s;
37     nQ_o <= not q_s;
38
39 end comport;

```

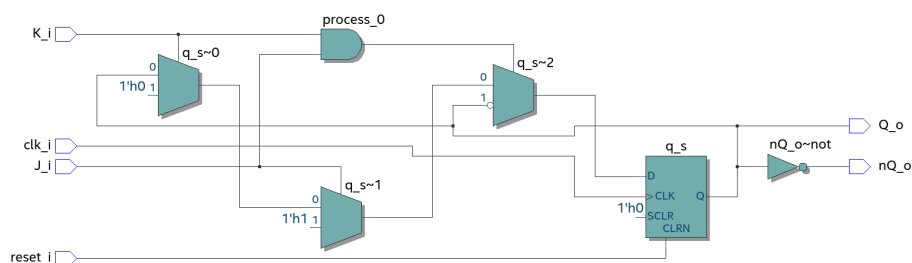


FIGURE 5 – Quartus Prime vue RTL flipflop_jk


```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity latch_en is
5     port(
6         en_i      : in      std_logic;
7         reset_i   : in      std_logic;
8         D_i       : in      std_logic;
9         Q_o       : out     std_logic
10    );
11 end latch_en ;
12
13 architecture comport of latch_en is
14
15 begin
16
17     process(D_i, en_i, reset_i)
18     begin
19         if reset_i = '1' then
20             Q_o <= '0';
21         elsif en_i = '1' then
22             Q_o <= D_i;
23         end if;
24     end process;
25
26 end comport;
```

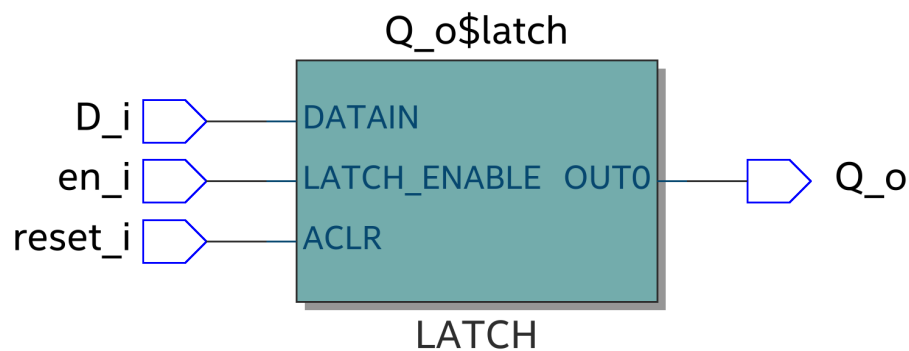


FIGURE 6 – Quartus Prime vue RTL latch_en

4 Simulation

```
# vsim -voptargs=""+acc"" work.dff_ar_tb
# Start time: 13:22:02 on Apr 22,2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading work.dff_ar_tb(test_bench)#1
# Loading work.dff_ar(comport)#1

VSIM 4> run -all
# ** Note: >> Debut de la simulation
#   Time: 0 ns   Iteration: 0   Instance: /dff_ar_tb
# ** Note: >>Nombre d'erreur détectée = 0
#   Time: 552 ns  Iteration: 0   Instance: /dff_ar_tb
# ** Note: >>Fin de la simulation
#   Time: 552 ns  Iteration: 0   Instance: /dff_ar_tb

VSIM 5>|
```

FIGURE 7 – Questa Sim dff_ar

```
# End time: 13:25:57 on Apr 22,2026, Elapsed time: 0:03:55
# Errors: 0, Warnings: 0
# vsim -voptargs=""+acc"" work.dff_en_tb
# Start time: 13:25:57 on Apr 22,2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading work.dff_en_tb(test_bench)#1
# Loading work.dff_en(comport)#1

VSIM 6> run -all
# ** Note: >> Debut de la simulation
#   Time: 0 ns   Iteration: 0   Instance: /dff_en_tb
# ** Note: >>Nombre d'erreur détectée = 0
#   Time: 652 ns  Iteration: 0   Instance: /dff_en_tb
# ** Note: >>Fin de la simulation
#   Time: 652 ns  Iteration: 0   Instance: /dff_en_tb

VSIM 7>
```

FIGURE 8 – Questa Sim dff_en

```
# End time: 15:38:23 on Apr 22,2026, Elapsed time: 0:02:33
# Errors: 0, Warnings: 0
# vsim -voptargs="+acc" work.flipflop_t_tb
# Start time: 15:38:23 on Apr 22,2026
# ** Note: (vsim-3813) Design is being optimized due to module recompilation...
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading work.flipflop_t_tb(test_bench)#1
# Loading work.flipflop_t(comport)#1
VSIM 30> run -all
# ** Note: >> Debut de la simulation
#   Time: 0 ns Iteration: 0 Instance: /flipflop_t_tb
# ** Note: >>Nombre d'erreur détectée = 0
#   Time: 652 ns Iteration: 0 Instance: /flipflop_t_tb
# ** Note: >>Fin de la simulation
#   Time: 652 ns Iteration: 0 Instance: /flipflop_t_tb
VSIM 31>
```

FIGURE 9 – Questa Sim flipflop_t

```
# End time: 15:48:06 on Apr 22,2026, Elapsed time: 0:06:35
# Errors: 0, Warnings: 0
# vsim -voptargs="+acc" work.flipflop_rs_tb
# Start time: 15:48:07 on Apr 22,2026
# ** Note: (vsim-3813) Design is being optimized due to module recompilation...
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading work.flipflop_rs_tb(test_bench)#1
# Loading work.flipflop_rs(comport)#1
VSIM 34> run -all
# ** Note: >> Debut de la simulation
#   Time: 0 ns Iteration: 0 Instance: /flipflop_rs_tb
# ** Note: Test du cas SR = 11, l'état de la sortie Q est indifferent
#   Time: 852 ns Iteration: 0 Instance: /flipflop_rs_tb
# ** Note: Verif asynchrone : état sortie Q pas vérifié
#   Time: 948 ns Iteration: 0 Instance: /flipflop_rs_tb
# ** Note: Verif synchrone : état sortie Q pas vérifié
#   Time: 998 ns Iteration: 0 Instance: /flipflop_rs_tb
# ** Note: >>Nombre d'erreur détectée = 0
#   Time: 1452 ns Iteration: 0 Instance: /flipflop_rs_tb
# ** Note: >>Fin de la simulation
#   Time: 1452 ns Iteration: 0 Instance: /flipflop_rs_tb
VSIM 35> |
```

FIGURE 10 – Questa Sim flipflop_rs

```
# End time: 16:00:39 on Apr 22,2026, Elapsed time: 0:12:32
# Errors: 0, Warnings: 0
# vsim -voptargs="" +acc"" work.flipflop_jk_tb
# Start time: 16:00:39 on Apr 22,2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading work.flipflop_jk_tb(test_bench)#1
# Loading work.flipflop_jk(comport)#1
VSIM 38> run -all
# ** Note: >> Debut de la simulation
#   Time: 0 ns Iteration: 0 Instance: /flipflop_jk_tb
# ** Note: >>Nombre d'erreur détectée = 0
#   Time: 1352 ns Iteration: 0 Instance: /flipflop_jk_tb
# ** Note: >>Fin de la simulation
#   Time: 1352 ns Iteration: 0 Instance: /flipflop_jk_tb
VSIM 39> |
```

FIGURE 11 – Questa Sim flipflop_jk

```
# End time: 14:07:40 on Apr 22,2026, Elapsed time: 0:21:08
# Errors: 0, Warnings: 0
# vsim -voptargs="" +acc"" work.latch_en_tb
# Start time: 14:07:40 on Apr 22,2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading std.standard
# Loading std.textio(body)
# Loading ieee.std_logic_1164(body)
# Loading work.latch_en_tb(test_bench)#1
# Loading work.latch_en(comport)#1
VSIM 11> run -all
# ** Note: >> Debut de la simulation
#   Time: 0 ns Iteration: 0 Instance: /latch_en_tb
# ** Note: >>Nombre d'erreur détectée = 0
#   Time: 752 ns Iteration: 0 Instance: /latch_en_tb
# ** Note: >>Fin de la simulation
#   Time: 752 ns Iteration: 0 Instance: /latch_en_tb
VSIM 12>
```

FIGURE 12 – Questa Sim latch_en

5 Conclusion

Comme nous avons pu le voir, les schémas et les vues RTL sont très similaires. Une différence notable est que des fois **Quartus** va préférer mettre des signaux à l'intérieur de l'élément mémoire crée, alors que d'autre fois il les met à l'extérieur comme des MUX ou portes logiques.

6 Signatures

Yverdon-les-Bains le 22 avril 2026

Léger Fabien